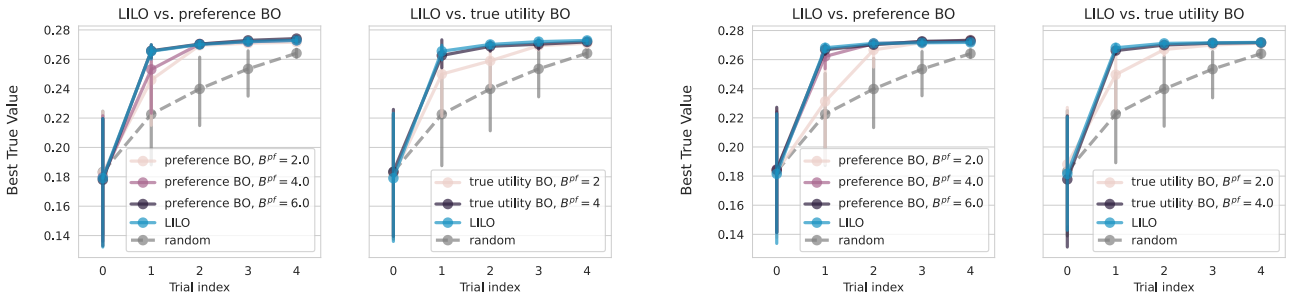
H. High-dimensional outcomes

Table 4. Overview of the text summarization benchmark. We use the CNN/DailyMail (Hermann et al., 2015; See et al., 2017) dataset. For each article a, a summarization LLM generates a summary $s = f(a, \mathbf{x})$ using decoding parameters $\mathbf{x} = (x_1, x_2, x_3, x_4)$ corresponding to temperature, top-p, maximum output length, and repetition penalty. Unlike in other environments, the realized outcome is the generated summary text, i.e. an unstructured high-dimensional object. For evaluation, each summary is associated with a latent utility based on ROUGE overlap (Lin, 2004) with the dataset reference summary r, normalized Flesch Reading Ease (Flesch, 1948), conciseness, non-repetition, and a completion penalty. For evaluation purposes, we design a latent utility function as:

$$u(s; a, r) = (w_{\text{R1}} q_{\text{R1}} + w_{\text{R2}} q_{\text{R2}} + w_{\text{RL}} q_{\text{RL}} + w_{\text{read}} q_{\text{read}} + w_{\text{conc}} q_{\text{conc}} + w_{\text{rep}} q_{\text{rep}}) \cdot \max(q_{\text{comp}}, 0.1),$$

where $q_{\text{R1}}, q_{\text{R2}}, q_{\text{RL}}$ are ROUGE-1/2/L F1 scores, q_{read} is normalized readability, q_{conc} is a soft conciseness score, q_{rep} is a non-repetition score, and $q_{\text{comp}} \in \{0, 1\}$ indicates whether the summary ends with a complete sentence. This experiment illustrates that `LILO` can be naturally extended to high-dimensional and unstructured outcomes. This is possible because `LILO` does not require an explicit model of the outcome space $\mathcal{Y}$. Instead, it only requires that the LLM can parse the realized outcomes and perform pairwise labeling of these outcomes given the DM's language feedback. This makes the framework naturally compatible with outcomes as text or images. Given the dependency of the black-box function on articles a we introduce two schemes for selecting the "training" articles used during optimization: **sampled** and **fixed**. This setup is summarized in the table below and in the Algorithms 6 and 7.

Component	Description
Task	Optimization of decoding parameters for summarization on the CNN/DailyMail dataset.
Input space	Decoding parameters $\mathbf{x} = (\texttt{temperature}, \texttt{top_p}, \texttt{max_new_tokens}, \texttt{repetition_penalty})$.
Outcome space	Generated summary text $s = f(a, \mathbf{x})$; the outcome is not a low-dimensional vector but a textual artifact.
ROUGE indicators	$q_{\text{R1}}, q_{\text{R2}}, q_{\text{RL}}$ are ROUGE-1/2/L F1 scores against the gold reference summary. ROUGE-1 measures unigram overlap, ROUGE-2 bigram overlap, and ROUGE-L overlap via the longest common subsequence.
Readability	q_{read}: normalized Flesch Reading Ease score, favoring shorter sentences and simpler vocabulary.
Conciseness	q_{conc}: soft preference for summaries of approximately paragraph length (around 90 words).
Non-repetition	q_{rep}: lexical diversity score rewarding non-redundant summaries.
Completion	$q_{\text{comp}} \in \{0, 1\}$: binary indicator of whether the summary ends with a complete sentence. Incomplete summaries are multiplicatively penalized via $\max(q_{\text{comp}}, 0.1)$.
Weights	$(w_{\text{R1}}, w_{\text{R2}}, w_{\text{RL}}, w_{\text{read}}, w_{\text{conc}}, w_{\text{rep}}) = (0.25, 0.25, 0.20, 0.15, 0.10, 0.05)$. This setting puts most emphasis on the overlap with the reference summary r.
Articles (**sampled**)	Each BO iteration uses a fresh minibatch of training articles; NL feedback is generated independently for each batch (see Prompt 13 and Algorithm 6).
Article (**fixed**)	The same training articles are reused across all BO iterations; NL feedback is accumulated additively: later feedback conditions on earlier feedback and the newly generated summaries, rather than regenerating the critique from scratch (see Prompt 14 and Algorithm 7).
NL feedback	For each article, a feedback-generating LLM produces NL critiques of candidate summaries; `LILO` converts these critiques into pairwise preference labels between candidate summaries (see Prompt 15).
Evaluation	Final performance is measured on a held-out set of unseen test articles.



(a) Sampled training articles. *(b)* Fixed training articles.

Figure 17. Optimization results on the text summarization example. In the sampled setting, each BO iteration uses a fresh batch of $N^{\text{art}} = 3$ training articles. In the fixed setting, all BO iterations reuse the same set of $N^{\text{art}} = 10$ training articles and the NL feedback is accumulated additively across rounds. In both cases, each trial evaluates $N^{\text{cfg}} = 4$ candidate decoding configurations, matching the dimensionality of the search space, and optimization is run for $T = 5$ batched trials, after which all methods have effectively saturated. Final evaluation is performed on a held-out test set of 20 unseen articles. With $N^{\text{cfg}} = 4$, each BO trial contains at most six unique pairwise comparisons and four direct utility evaluations. Accordingly, preference BO is evaluated with feedback budgets $B^{\text{pf}} \in \{2, 4, 6\}$, while true-utility BO is evaluated with $B^{\text{pf}} \in \{2, 4\}$. We also report a random acquisition baseline. Across both the sampled and fixed settings, `LILO` matches the strongest baselines when those baselines are given the maximum amount of oracle feedback. This is notable because preference BO with $B^{\text{pf}} = 6$ and true-utility BO with $B^{\text{pf}} = 4$ effectively represent upper bounds on the amount of information available per trial. For smaller feedback budgets, `LILO` typically improves more rapidly during the first one to two optimization rounds. Both `LILO` and the BO baselines substantially outperform random search. Overall, these results show that `LILO` can optimize over high-dimensional textual outcomes without explicitly modeling the outcome space itself. Instead, it relies only on natural-language critiques and pairwise judgments over generated summaries and achieves optimization performance comparable to methods with direct access to the oracle latent utility function.

Algorithm 6: `LILO` with **sampled** articles

Input: Number of BO trials T, number of candidate configurations per trial N^{cfg}, number of articles per trial N^{art}, training article pool $\mathcal{D}_{\text{train}}$, test article set $\mathcal{D}_{\text{test}}$
Output: Proxy model M_T and final incumbent decoding configuration
$D_0^{\text{exp}} \leftarrow \emptyset$
$D_0 \leftarrow \emptyset$
for $n = 1$ **to** T **do**
  if $n = 1$ **then**
    $\{x_i\}_{i=1}^{N^{\text{cfg}}} \sim \text{Uniform}(\mathcal{X})$
  else
    $\{x_i\}_{i=1}^{N^{\text{cfg}}} \sim \text{acqf}(\mathcal{X}; M_{n-1})$
  end
  $\{a_j\}_{j=1}^{N^{\text{art}}} \sim \mathcal{D}_{\text{train}}$ `// fresh article batch at each trial`
  foreach $x_i \in \{x_i\}_{i=1}^{N^{\text{cfg}}}$ *and* $a_j \in \{a_j\}_{j=1}^{N^{\text{art}}}$ **do**
    Generate summary $s_{ij} = f(a_j, x_i)$
    Compute summary indicators q_{ij} and latent utility u_{ij}
  end
  $D_n^{\text{exp}} \leftarrow D_{n-1}^{\text{exp}} \cup \{(x_i, a_j, s_{ij}, q_{ij})\}_{i=1,j=1}^{N^{\text{cfg}},N^{\text{art}}}$
  foreach $a_j \in \{a_j\}_{j=1}^{N^{\text{art}}}$ **do**
    $F_j^{(n)} \leftarrow \text{DM.get_feedback}\Big(a_j, \{s_{ij}, q_{ij}\}_{i=1}^{N^{\text{cfg}}}\Big)$
    $P_j^{(n)} \leftarrow \text{LILO.get_pair_labels}\Big(a_j, r_j, \{s_{ij}\}_{i=1}^{N^{\text{cfg}}}, F_j^{(n)}\Big)$
  end
  $D_n \leftarrow D_{n-1} \cup \{(a_j, P_j^{(n)})\}_{j=1}^{N^{\text{art}}}$
  $M_n \leftarrow \text{fit proxy}(D_n^{\text{exp}}, D_n)$
end

Figure 18. `LILO` with **sampled** articles. At each BO trial, a fresh batch of training articles is sampled, candidate summaries are generated, natural-language feedback is collected, pairwise labels are inferred, and the proxy model is updated.

Algorithm 7: `LILO` with **fixed** articles

Input: Number of BO trials T, number of candidate configurations per trial N^{cfg}, fixed training article set $\mathcal{A}_{\text{train}} = \{a_j\}_{j=1}^{N^{\text{art}}}$, test article set $\mathcal{D}_{\text{test}}$
Output: Proxy model M_T and final incumbent decoding configuration
$D_0^{\text{exp}} \leftarrow \emptyset$
$D_0 \leftarrow \emptyset$
$\mathcal{F}_j \leftarrow \emptyset$ for all $a_j \in \mathcal{A}_{\text{train}}$
for $n = 1$ **to** T **do**
  if $n = 1$ **then**
    $\{x_i\}_{i=1}^{N^{\text{cfg}}} \sim \text{Uniform}(\mathcal{X})$
  else
    $\{x_i\}_{i=1}^{N^{\text{cfg}}} \sim \text{acqf}(\mathcal{X}; M_{n-1})$
  end
  foreach $x_i \in \{x_i\}_{i=1}^{N^{\text{cfg}}}$ *and* $a_j \in \mathcal{A}_{train}$ **do**
    Generate summary $s_{ij}^{(n)} = f(a_j, x_i)$
    Compute summary indicators $q_{ij}^{(n)}$ and latent utility $u_{ij}^{(n)}$
  end
  $D_n^{\text{exp}} \leftarrow D_{n-1}^{\text{exp}} \cup \{(x_i, a_j, s_{ij}^{(n)}, q_{ij}^{(n)})\}_{i=1, a_j \in \mathcal{A}_{\text{train}}}$
  foreach $a_j \in \mathcal{A}_{train}$ **do**
    if $n = 1$ **then**
      $F_j^{(1)} \leftarrow \text{DM.get_feedback}\Big(a_j, \{s_{ij}^{(1)}, q_{ij}^{(1)}\}_{i=1}^{N^{\text{cfg}}}\Big)$
    else
      $F_j^{(n)} \leftarrow \text{DM.get_incremental_feedback}\Big(a_j, \mathcal{F}_j, \{s_{ij}^{(n)}, q_{ij}^{(n)}\}_{i=1}^{N^{\text{cfg}}}\Big)$
    end
    $\mathcal{F}_j \leftarrow \mathcal{F}_j \oplus F_j^{(n)}$ `// additive feedback accumulation`
    $P_j^{(n)} \leftarrow \text{LILO.get_pair_labels}\Big(a_j, r_j, \{s_{ij}^{(\ell)}\}_{i, \ell \leq n}, \mathcal{F}_j\Big)$
  end
  $D_n \leftarrow \{(a_j, P_j^{(n)})\}_{a_j \in \mathcal{A}_{\text{train}}}$
  $M_n \leftarrow \text{fit proxy}(D_n^{\text{exp}}, D_n)$
end
Evaluate the final candidate configurations on $\mathcal{D}_{\text{test}}$

Figure 19. `LILO` with **fixed** training articles. The same set of N^{art} training articles is reused across all BO trials. At each trial, N^{cfg} decoding configurations are evaluated on this fixed article set, natural-language feedback is accumulated additively across rounds, `LILO`'s pairwise preferences are re-generated for all possible pairs of summaries per article based on the updated feedback, and the proxy model M_n is refit.

Feedback generation in `DM.get_feedback`

```
You are simulating a human decision-maker giving feedback on candidate summaries of a
news article.

Your preferences:
- Capturing the key facts from the article is my top priority, coverage and
faithfulness matter most
- I also value readability and conciseness. A good summary should be tight
   roughly a short paragraph. Extra details beyond the core facts hurt more than they
help.
- I dislike repetitive, verbose, or incomplete summaries.

You are shown:
1. The original article
2. Several candidate summaries
3. Some automatically computed quality indicators for each summary

Important:
- Use the summaries themselves as the main signal.
- Use the numeric indicators as supporting evidence.
- Make sure your preferences align with the "overall_utility" score (higher is better,
 range [0, 1]).
- Penalize summaries that finish mid-sentence (the binary "completion" score).
Incomplete summaries receive a high penalty.
- Do not mention any scores explicitly in your feedback.

Instructions:
- First, briefly describe what properties matter most to you for a good summary of
this particular article.
- When evaluating informativeness, focus on whether summaries capture the most
essential facts  the kind of information that would appear in a short headline-style
reference summary. Extra details from the article that go beyond the core facts are
less important to you.
- Note any trade-offs you see between factual coverage of core points, readability,
and conciseness.
- Give one or two concrete examples to illustrate. If some summaries are close in
quality but differ in specific ways, mention those differences  even small ones.
- If you have a slight preference between two similar candidates, say so briefly and
explain why.
- Be concise. Do not write more than a short paragraph.

Original article:
\"\"\"
{article}
\"\"\"

Candidate summaries and indicators:
{summaries_block}

Please provide feedback on the summaries. Describe what properties you found most
important for summarizing this particular article, and what trade-offs you noticed
across the batch.

Return plain text only.
```

Prompt 13. NL feedback generating prompt in `DM.get_feedback` for text summarization

Feedback generation in `DM.get_incremental_feedback`

```
You are simulating a human decision-maker giving iterative feedback on candidate
summaries of a news article.

Your preferences:
- Capturing the key facts from the article is my top priority, coverage and
faithfulness matter most
- I also value readability and conciseness. A good summary should be tight
   roughly a short paragraph. Extra details beyond the core facts hurt more than they
help.
- I dislike repetitive, verbose, or incomplete summaries.

You previously reviewed several candidate summaries and provided this feedback:
\"\"\"
{previous_feedback}
\"\"\"

Your previous best was candidate_{best_prev_idx}:
\"\"\"
{best_prev_summary}
\"\"\"

You are now shown {len(new_summaries)} new candidate summaries for the same article.

Important:
- Use the summaries themselves as the main signal.
- Use the numeric indicators as supporting evidence.
- Make sure your preferences align with the "overall_utility" score (higher is better,
 range [0, 1]).
- Penalize summaries that finish mid-sentence (the binary "completion" score).
Incomplete summaries receive a high penalty.
- Do not mention any scores explicitly in your feedback.

Instructions:
- Mention explicitly at most two new candidate summaries.
- Compare those new candidates to your previous best (candidate_{best_prev_idx}).
- Be specific about what improved or regressed.
- Be concise. Do not write more than a short paragraph.

Original article:
\"\"\"
{article}
\"\"\"

New candidate summaries and indicators:
{new_summaries_block}

Provide a concise update to your assessment. Focus on what changed.

Return plain text only.
```

Prompt 14. NL feedback generating prompt in `DM.get_incremental_feedback` for text summarization

LILO pairwise preference labeling in LILO.get_pair_labels

```
You are an expert judge inferring a decision maker's preferences over candidate
summaries.

Your task is to compare candidate summaries PAIRWISE and decide which summary the
decision maker would prefer, based on:
1. the original article,
2. the reference summary,
3. the expert feedback describing the decision maker's preferences.

Important instructions:
- Judge from the perspective of the decision maker described in the feedback.
- If the feedback explicitly praises or criticizes a specific candidate, treat that as
 a strong signal  it directly reflects the decision maker's preferences.
- Use the original article and reference summary to assess coverage, faithfulness,
readability, conciseness, and completeness.
- Prefer summaries that better match the stated preferences in the feedback.
- For each pair, choose exactly one winner. Do not output ties.
- Compare each pair independently.
- Keep your reasoning short, specific, and grounded in the actual summaries and the
expert feedback.

Original article:
\"\"\"
{article}
\"\"\"

Reference summary:
\"\"\"
{ref_summary}
\"\"\"

Expert feedback expressing the decision maker's preferences:
\"\"\"
{feedback}
\"\"\"

Candidate summaries:
{candidate_block}

You must produce pairwise judgments for ALL of the following pairs:
{pair_block}

Return pairwise comparisons between ALL unique pairs of candidate summaries.
Provide your response in the following jsonl format (remember about the jsonl header!).

IMPORTANT: The "winner" field must be the candidate ID string (e.g. "candidate_0"), NOT
 a numeric index.

```jsonl
{{
"option_0": "candidate_{first_i}",
"option_1": "candidate_{first_j}",
"reasoning": "Short reasoning comparing the two candidates from the decision maker's
perspective.",
"winner": "candidate_xx"
}}
...
```
```

Prompt 15. Prompt to obtain pairwise preference labels between summaries in LILO.get_pair_labels